

Alzikrayat Architecture Report

1. Introduction

Alzikrayat is a photo-sharing web application developed for the Advanced Web Technologies course. Its purpose is to demonstrate how dynamic web pages can be generated from relational database content without depending on a backend framework. The application lets a visitor browse photographs and read their comments. After registration and login, a user can upload a photograph, add comments to any photograph, and delete only photographs that belong to that user.

The implementation uses PHP 8, MySQL, PDO, HTML5, vanilla JavaScript, Bootstrap 5, and Apache URL rewriting. The design intentionally stays small enough for a student to understand. There is no ORM, framework router, service layer, or dependency injection container. The main architecture combines the Model-View-Controller pattern with a physical three-tier structure.

2. Model-View-Controller Architecture

Model-View-Controller, usually called MVC, separates an application into three logical responsibilities. This prevents database work, request decisions, and page markup from becoming one large PHP file.

The Models are `User`, `Photo`, and `Comment`. Each model extends the small base `Model` class and receives a PDO database connection. A model contains handwritten SQL and validation rules connected to its data. For example, `User` checks registration values, finds an account by email, creates a password hash, and counts registered users. `Photo` loads joined gallery data, finds a single photograph, stores metadata, and performs an ownership-restricted delete. `Comment` stores and retrieves comments associated with users and photographs.

The Controllers are `AuthController`, `PhotoController`, and `CommentController`. Controllers receive an HTTP request through the router, decide what work is required, call models, and choose a view or redirect. They do not contain page markup. `AuthController` manages register, login, logout, sessions, and the last-login cookie. `PhotoController` manages the home page, About Us, gallery, uploads, photo details, and deletion. `CommentController` checks authentication and stores a comment for an existing photograph.

The Views are PHP templates inside `views/`. Their job is presentation. They contain HTML5 and Bootstrap classes with only small PHP statements for escaped output and conditional display. Common header, navbar, flash message, footer, stylesheet, and script references are shared through layout templates. Views never run SQL. Dynamic values are escaped through the `e()` helper before being included in HTML.

The base `Controller` provides only operations shared by all controllers: rendering a view, redirecting, requiring authentication, checking CSRF tokens, and returning a not-found page. This avoids repetition while keeping the control flow visible.

3. Three-Tier Architecture

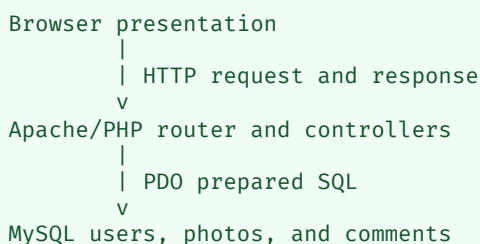
Three-tier architecture describes the wider physical boundaries of the system. It coexists with MVC rather than replacing it.

The Presentation Tier runs in the user's browser. It includes the rendered HTML5 pages, Bootstrap responsive layout, the custom CSS file, and vanilla JavaScript validation and interactions. The browser displays forms and photographs, changes gallery layouts, previews an upload, and gives immediate validation feedback. The browser cannot be trusted for security because users can disable or change JavaScript.

The Application or Business Tier runs in Apache and PHP. `public/index.php`, `Router.php`, the controllers, PHP validation, authentication checks, ownership checks, and upload processing belong here. This tier receives requests, applies the application rules, and prepares safe responses. It is the only tier that knows both what the browser requested and which model operation is allowed.

The Data Tier is MySQL together with PDO connection and model query code. `config/database.php` opens the database connection. The models use manually written prepared SQL. Foreign keys maintain relationships even if application code makes a mistake. The browser has no direct database access and never receives database credentials.

This separation produces the following boundary:



4. Front Controller and Manual Router

Apache uses `public/.htaccess` to send every request that is not a real public file or directory to `public/index.php`. This file is the single front controller. It configures secure session-cookie options, starts the session, loads classes, registers application routes, and dispatches the current request.

The custom `Router` stores each route as an HTTP method, a path pattern, and a handler. GET and POST routes are registered separately. During dispatch, the router removes an Apache subdirectory prefix, removes a trailing slash, and compares the requested HTTP method with each route.

For a parameterized route such as `/photo/{id}`, `compilePattern()` splits the route around placeholders. Static parts are protected with `preg_quote()`. The `{id}` placeholder becomes a named regular-expression group that captures characters until the next slash. Anchors are added so the complete path must match. The resulting idea is equivalent to:

```
#^/photo/(?P<id>[^/+]$)#
```

When the browser requests `/photo/25`, the named group captures 25. The router decodes the value and calls `PhotoController::show("25")`. The controller then accepts only positive digits before changing the string to an integer. If no registered method and path match, the router sets HTTP status 404 and calls the common not-found renderer.

The router does not use a framework, third-party package, or Apache rule for individual actions. Apache only forwards the request. All application matching and parameter extraction are visible in `core/Router.php`.

5. Database Design

The MySQL schema contains the three normalized entities required by the specification.

The `users` table stores an auto-incrementing ID, required first and last names, a unique email, a password hash, and optional location, biography, and occupation. Profile information is stored once with its user rather than repeated on photographs or comments.

The `photos` table stores an auto-incrementing ID, its owner through `user_id`, the safe physical filename, required title, optional description, and an automatic timestamp. Author names are not copied into this table. A join reads current user information when a page needs it.

The `comments` table stores an auto-incrementing ID, `photo_id`, `user_id`, required comment text, and an automatic timestamp. This creates two many-to-one relationships: many comments can belong to one photograph, and many comments can be written by one user.

All foreign keys use InnoDB and cascade deletion. Deleting a user removes that user's photo metadata and comments. Deleting a photo removes its comments. Relationship columns and common date ordering columns have indexes. The schema uses `utf8mb4` so international text can be stored correctly.

All model queries are written by hand. External values are represented by named placeholders and bound through PDO. PDO emulated preparation is disabled, which asks MySQL to prepare supported statements natively.

6. Authentication, Sessions, and Cookie State

Registration passes through three validation levels. PHP confirms required names, letter-only names, length limits, valid and unique email, matching passwords, and limits for optional fields. `password_hash()` converts the password into a one-way hash. Only that hash is inserted in MySQL.

During login, the application finds the normalized email through a prepared query. `password_verify()` safely checks the supplied password against the stored hash. The failure message does not reveal whether the email or password was wrong. A successful login regenerates the session identifier and stores only the user's ID and first name in session state. These two values are enough for authorization and the required navbar greeting.

Logout is a protected POST action. It checks the CSRF token, removes the user from the session, and regenerates the session identifier. The independent last-login cookie remains. After every successful login, the server stores an exact timestamp in the `last_login` cookie with a seven-day expiry, `HttpOnly`, `SameSite=Lax`, and HTTPS-only transmission when HTTPS is active. At the next visit to the login page, a correctly formatted cookie is displayed as the previous login from that browser.

7. Photo and Comment Request Flow

An authenticated photo upload begins in `views/photos/create.php`. HTML restricts the input type and size, while JavaScript provides quick feedback and a local preview. PHP then checks the title, description, upload status, 5 MB limit, real MIME type through Fileinfo, and valid image metadata through `getimagesize()`. The original filename is not trusted. Random bytes generate a new server filename with an extension selected from the detected MIME type.

The controller moves the file into `public/images/uploads/` and asks `Photo::create()` to store metadata. If the database insert fails, the new file is removed. For deletion, the controller loads the photo and compares `user_id` with the authenticated ID. The model repeats this rule in the final SQL delete. The file receives a temporary name while the database

delete runs; it is restored if the database operation fails and removed after a successful delete. Comments disappear through the database cascade.

A comment POST request contains the current CSRF token and text. `CommentController` requires authentication, accepts only a positive photo ID, confirms that the photograph exists, and validates non-empty text up to 1000 characters. `Comment::create()` binds the photo ID, current user ID, and text. The controller redirects to the comments section of the detail page, where a new database read immediately includes the new comment.

8. Security and Validation

The application uses defense in depth. HTML5 validation gives standard browser constraints. Custom JavaScript checks names, email, password confirmation, trimmed text, image type, and size. PHP repeats and controls every security-relevant decision because a crafted HTTP request can bypass the browser.

SQL injection is prevented through prepared statements. User values never become SQL structure. Cross-site scripting is prevented at output time: the `e()` helper calls `htmlspecialchars()` with quote handling and UTF-8 substitution. Line breaks are added only after escaping comment and description text.

CSRF protection uses a random 32-byte session token and `hash_equals()`. All actions that change state use POST and require this token. Upload files receive unpredictable names and only verified raster-image MIME types are accepted. Route IDs are validated before queries. Exceptions are logged on the server while a generic 500 page is shown to users.

9. Responsive Interface and Novelty

Bootstrap 5 supplies the responsive grid, navbar, forms, alerts, and buttons. Custom CSS adds a consistent green and amber memory-inspired visual identity. Gallery cards change from four or three columns on wide screens to two columns on tablets and one column on phones. List view presents a horizontal card where space permits. Additional orientation media queries adjust the hero area for landscape phones and portrait tablets. Controls have visible keyboard focus, meaningful labels, adequate touch size, and reduced-motion support.

The one novelty feature is Memory Mood. On a photo detail page, Original, Warm, and Monochrome controls apply reversible CSS filter combinations through vanilla JavaScript. The physical upload and database metadata never change. This makes the feature immediate and safe while demonstrating client-side image presentation.

10. Conclusion

Alzikrayat meets the project purpose with direct code that can be traced from a URL to a controller, model query, and rendered view. MVC keeps logical duties separate, while the three-tier boundary protects the database from direct browser access. The manual router, normalized schema, prepared SQL, session authentication, seven-day cookie, ownership enforcement, upload checks, output escaping, and responsive presentation demonstrate the concepts required by the course without hiding them behind a framework.